



**UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO**  
**FACULTAD DE INGENIERÍA**  
**INGENIERÍA EN COMPUTACIÓN**

**Laboratorio de Organización y Arquitectura de Computadoras**

**Reporte - Diseño de Máquinas de Estado**

**INTEGRANTES EQUIPO 8:**

- Saldaña Navarrete Andrea - *Grupo Teoria: 1*
- Tepal Briseño Hansel Yael - *Grupo Teoria: 1*
- Ugartechea González Luis Antonio - *Grupo Teoria: 1*

**NOMBRE DEL PROFESOR:** Ing. Diana Karen Pérez Pérez

**GRUPO:** 06

**SEMESTRE:** 2027-1

**FECHA DE ENTREGA:** 10 de septiembre del 2026

## Objetivo

Analizar, diseñar e implementar una máquina de estados finitos a partir de una carta ASM, identificando el funcionamiento de los flip-flops tipo D como elementos de memoria. Obtener las funciones de estado siguiente y salida mediante el llenado de memoria y mapas de Karnaugh, para posteriormente comprobar su funcionamiento utilizando la FPGA DE10-Lite.

## Introducción

## Desarrollo

### Parte I. Análisis y diseño de la máquina de estados

A partir de la carta ASM proporcionada en la Figura 1:

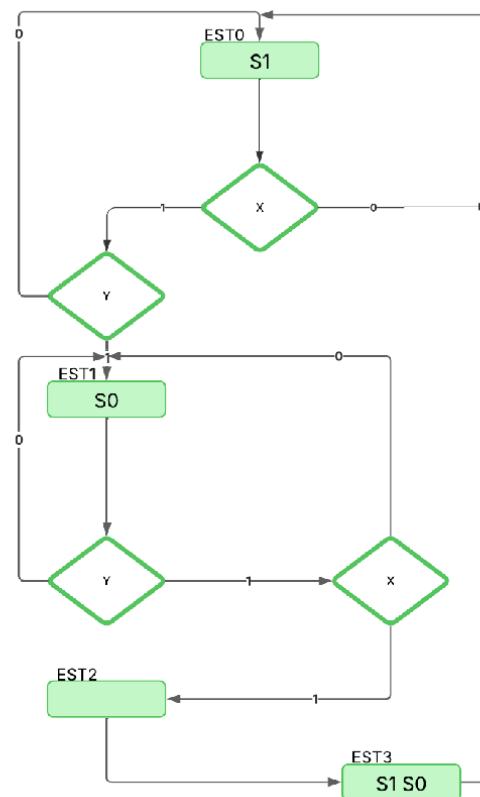


Figure 1: Carta ASM.

Realizar lo siguiente:

### 1. Identifique los estados, entradas y salidas de la máquina.

- **Estados:** El sistema cuenta con 4 estados definidos en los rectángulos: EST0, EST1, EST2 y EST3.
- **Entradas:** Son dos entradas:  $X$  e  $Y$ .
- **Salidas:** Dentro de los bloques de estado se activan las señales  $S_1$  y  $S_0$ .
  - En EST0: Se activa  $S_1$ .
  - En EST1: Se activa  $S_0$ .
  - En EST2: No hay salidas activas.
  - En EST3: Se activan  $S_1$  y  $S_0$ .

### 2. Determine el número de bits necesarios para representar los estados y proponga una codificación.

Dado que se tienen 4 estados, se calcula el número de bits  $n$  necesarios usando  $2^n \geq 4$ , resultando en  $n = 2$  bits. Se utilizarán dos variables de estado ( $E_1$  y  $E_0$ ) que corresponden a dos Flip-Flops tipo D. La codificación binaria propuesta es:

- EST0 = 00
- EST1 = 01
- EST2 = 10
- EST3 = 11

### 3. Obtenga la tabla de transiciones, considerando el estado actual y las entradas del sistema.

A partir de la carta ASM, la tabla de transiciones muestra el estado siguiente ( $Q_1(t+1)Q_0(t+1)$ ) en función del estado actual ( $E_1(t)E_0(t)$ ) y las combinaciones de las entradas  $X$  e  $Y$ :

| Estado Actual<br>$E_1E_0$ | Entradas $XY$ |    |    |    |
|---------------------------|---------------|----|----|----|
|                           | 00            | 01 | 11 | 10 |
| 00 (EST0)                 | 11            | 11 | 01 | 00 |
| 01 (EST1)                 | 01            | 11 | 10 | 01 |
| 10 (EST2)                 | 11            | 11 | 11 | 11 |
| 11 (EST3)                 | 00            | 00 | 00 | 00 |

Table 1: Tabla de transiciones.

4. Determine para cada combinación: Estado actual, Entradas, Estado siguiente, Salidas.

Los incisos 3 y 4 se resuelven de manera conjunta en la siguiente tabla de verdad general y más completa de la máquina de estados:

| Estado Actual |          | Entradas |     | Estado Siguiente |            | Salidas |       |
|---------------|----------|----------|-----|------------------|------------|---------|-------|
| $E_1(t)$      | $E_0(t)$ | $X$      | $Y$ | $Q_1(t+1)$       | $Q_0(t+1)$ | $S_1$   | $S_0$ |
| 0             | 0        | 0        | 0   | 0                | 0          | 1       | 0     |
| 0             | 0        | 0        | 1   | 0                | 0          | 1       | 0     |
| 0             | 0        | 1        | 0   | 0                | 0          | 1       | 0     |
| 0             | 0        | 1        | 1   | 0                | 1          | 1       | 0     |
| 0             | 1        | 0        | 0   | 0                | 1          | 0       | 1     |
| 0             | 1        | 0        | 1   | 0                | 1          | 0       | 1     |
| 0             | 1        | 1        | 0   | 0                | 1          | 0       | 1     |
| 0             | 1        | 1        | 1   | 1                | 0          | 0       | 1     |
| 1             | 0        | 0        | 0   | 1                | 1          | 0       | 0     |
| 1             | 0        | 0        | 1   | 1                | 1          | 0       | 0     |
| 1             | 0        | 1        | 0   | 1                | 1          | 0       | 0     |
| 1             | 0        | 1        | 1   | 1                | 1          | 0       | 0     |
| 1             | 1        | 0        | 0   | 0                | 0          | 1       | 1     |
| 1             | 1        | 0        | 1   | 0                | 0          | 1       | 1     |
| 1             | 1        | 1        | 0   | 0                | 0          | 1       | 1     |
| 1             | 1        | 1        | 1   | 0                | 0          | 1       | 1     |

Table 2: Tabla de transiciones y salidas de la máquina de estados.

5. Obtenga los mapas de Karnaugh correspondientes a cada entrada de los flip-flops tipo D y a las señales de salida.

| $E_1E_0 \backslash XY$ | 00 | 01 | 11 | 10 |
|------------------------|----|----|----|----|
| 00                     | 0  | 0  | 1  | 0  |
| 01                     | 1  | 1  | 0  | 1  |
| 11                     | 0  | 0  | 0  | 0  |
| 10                     | 1  | 1  | 1  | 1  |

Table 3: Mapa de Karnaugh para  $Q_1(t+1)$ .

| $E_1E_0 \backslash XY$ | 00 | 01 | 11 | 10 |
|------------------------|----|----|----|----|
| 00                     | 0  | 0  | 0  | 0  |
| 01                     | 0  | 0  | 1  | 0  |
| 11                     | 0  | 0  | 0  | 0  |
| 10                     | 1  | 1  | 1  | 1  |

Table 4: Mapa de Karnaugh para  $Q_0(t+1)$ .

| $E_1E_0 \backslash XY$ | 00 | 01 | 11 | 10 |
|------------------------|----|----|----|----|
| 00                     | 1  | 1  | 1  | 1  |
| 01                     | 0  | 0  | 0  | 0  |
| 11                     | 1  | 1  | 1  | 1  |
| 10                     | 0  | 0  | 0  | 0  |

Table 5: Mapa de Karnaugh para  $S_1$ .

| $E_1E_0 \backslash XY$ | 00 | 01 | 11 | 10 |
|------------------------|----|----|----|----|
| 00                     | 0  | 0  | 0  | 0  |
| 01                     | 1  | 1  | 1  | 1  |
| 11                     | 1  | 1  | 1  | 1  |
| 10                     | 0  | 0  | 0  | 0  |

Table 6: Mapa de Karnaugh para  $S_0$ .

## 6. Simplifique las funciones booleanas.

De los mapas de Karnaugh anteriores se deducen las siguientes expresiones booleanas simplificadas (suma de productos):

$$Q_1(t+1) = E_1\overline{E_0} + \overline{E_1}E_0\overline{X} + \overline{E_1}E_0\overline{Y} + \overline{E_0}XY$$

$$Q_0(t+1) = E_1\overline{E_0} + \overline{E_1}E_0XY$$

$$S_1 = \overline{E_1}\overline{E_0} + E_1E_0$$

$$S_0 = E_0$$

7. Dibuje el circuito resultante utilizando: Flip-flop tipo D, compuertas y lógica.

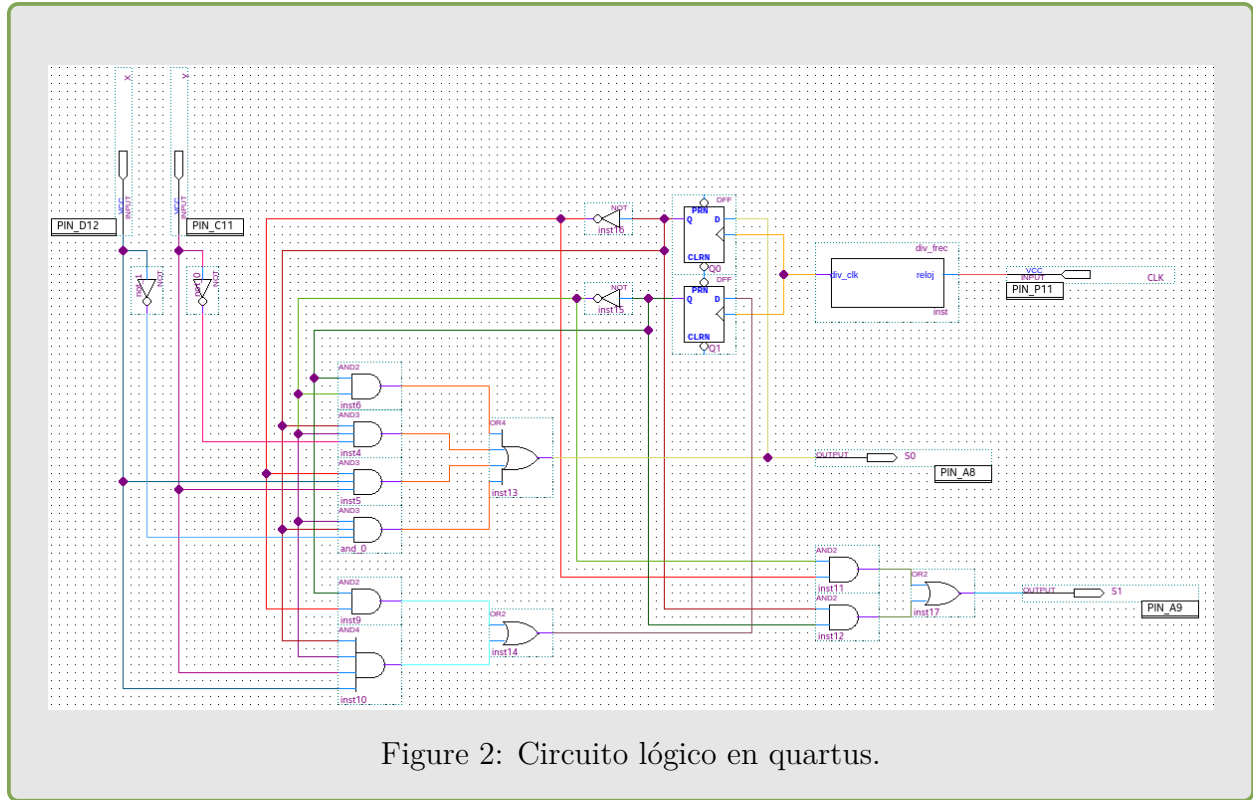


Figure 2: Circuito lógico en quartus.

## Parte II. Implementación en Quartus y FPGA DE10-Lite.

### 1. Implementación en Block Diagram/Schematic File

Se implementó el mismo circuito mostrado en la figura 2 en el software Quartus, utilizando un archivo de diagrama de bloques/esquemático. Se utilizaron Flip-Flops tipo D y compuertas lógicas para representar las funciones booleanas obtenidas previamente. A continuación se muestra la compilación y programación del circuito en la FPGA DE10-Lite.

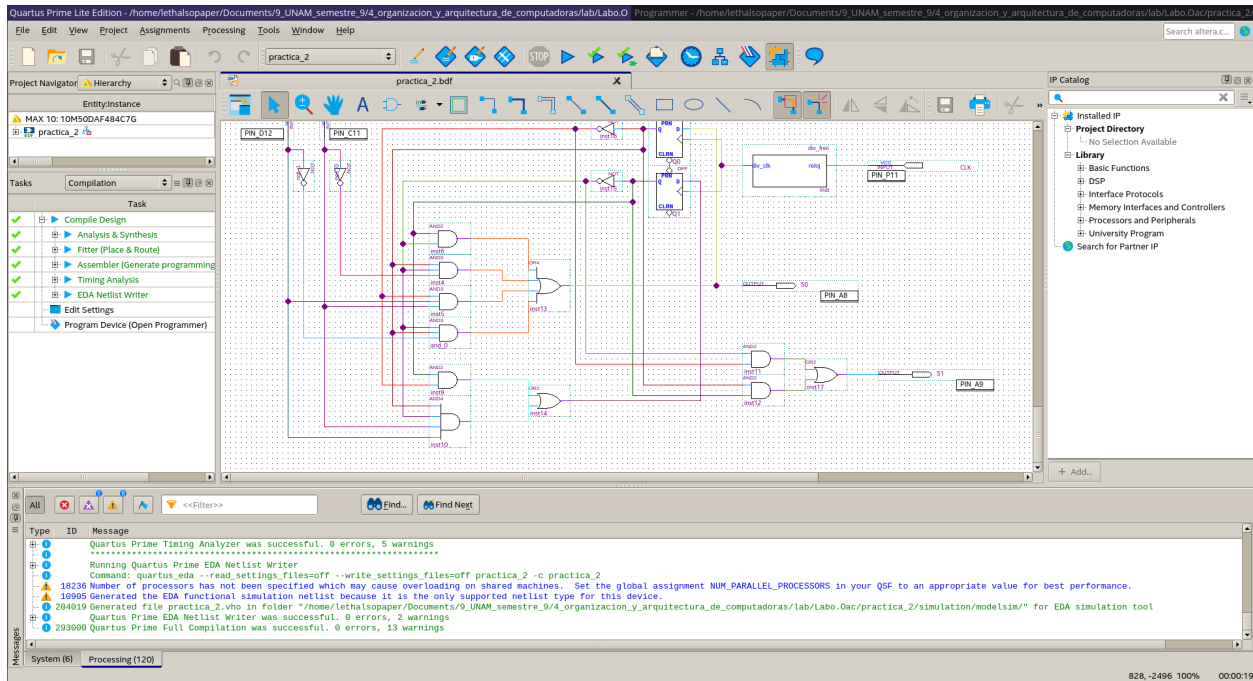


Figure 3: Compilación del circuito en Quartus.

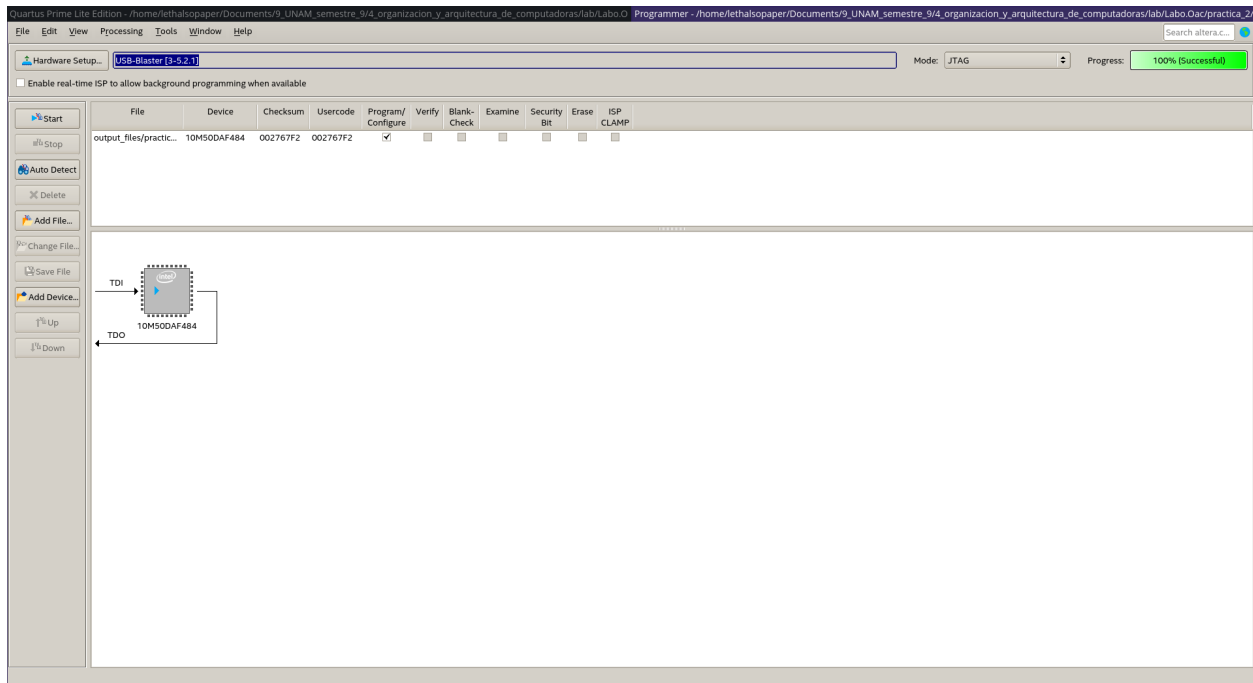


Figure 4: Programación del circuito en la FPGA DE10-Lite.

## 2. Implementación en VHDL

La máquina de estados se implementó en VHDL mediante un divisor de frecuencia, dos registros de estado tipo D, lógica combinacional para el estado siguiente y las salidas conectadas a los LED de la FPGA.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity practica_2 is
6      port (
7          clock : in  std_logic;
8          reset  : in  std_logic;
9          SW     : in  std_logic_vector(1 downto 0);
10         LEDR   : out std_logic_vector(1 downto 0)
11     );
12 end entity practica_2;
13
14 architecture rtl of practica_2 is
15     signal Q0      : std_logic_vector(1 downto 0) := "00";
16     signal Q1      : std_logic_vector(1 downto 0);
17     signal x       : std_logic;
18     signal y       : std_logic;
19     signal clk_slow : std_logic := '0';

```

```

20     signal rebote      : std_logic := '1';
21     signal pulso       : std_logic;
22 begin
23     x <= SW(0);
24     y <= SW(1);
25
26     -- Divisor de frecuencia
27     process (clock)
28         variable delay : integer range 0 to 50000000 := 0;
29         constant MAX_DELAY : integer := 2500000;
30     begin
31         if rising_edge(clock) then
32             if (delay < MAX_DELAY) then
33                 delay := delay + 1;
34             else
35                 clk_slow <= not clk_slow;
36                 delay := 0;
37             end if;
38         end if;
39     end process;
40
41     -- Entradas D de los flip-flops de Q0 (Estado Siguiente)
42     Q1(1) <= (Q0(1) and not Q0(0)) or
43             ((not Q0(1)) and Q0(0) and not x) or
44             ((not Q0(1)) and Q0(0) and not y) or
45             ((not Q0(0)) and x and y);
46
47     Q1(0) <= (Q0(1) and not Q0(0)) or
48             ((not Q0(1)) and Q0(0) and x and y);
49
50     -- Lógica anti-rebote (Detector de flanco de bajada)
51     -- Genera un '1' de duración de un solo ciclo de clk_slow
52     -- cuando el botón reset es presionado ('0').
53     pulso <= '1' when (reset = '0' and rebote = '1') else '0';
54
55     -- Memoria de la máquina de estados
56
57     process (clk_slow)
58     begin
59         if rising_edge(clk_slow) then
60             rebote <= reset;
61             if pulso = '1' then
62                 Q0 <= "00";
63             else
64                 Q0 <= Q1;
65             end if;
66         end if;
67     end process;

```

```

68      -- Salidas
69      LEDR(1 downto 0) <= Q0;
70
71  end architecture rtl;
72

```

Listing 1: Implementación de la máquina de estados en VHDL.

El proceso de compilación y programación del circuito en la FPGA DE10-Lite se realizó de manera similar a la implementación en Block Diagram/Schematic File, obteniendo los mismos resultados.

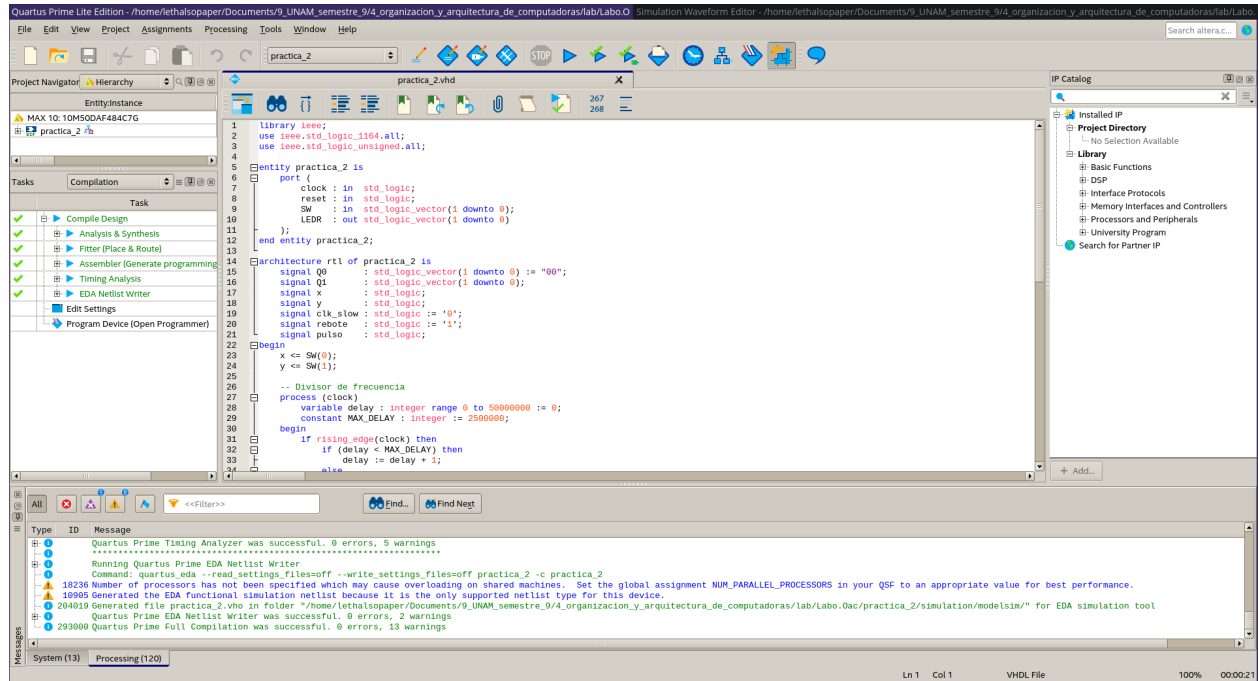


Figure 5: Compilación del circuito en Quartus.

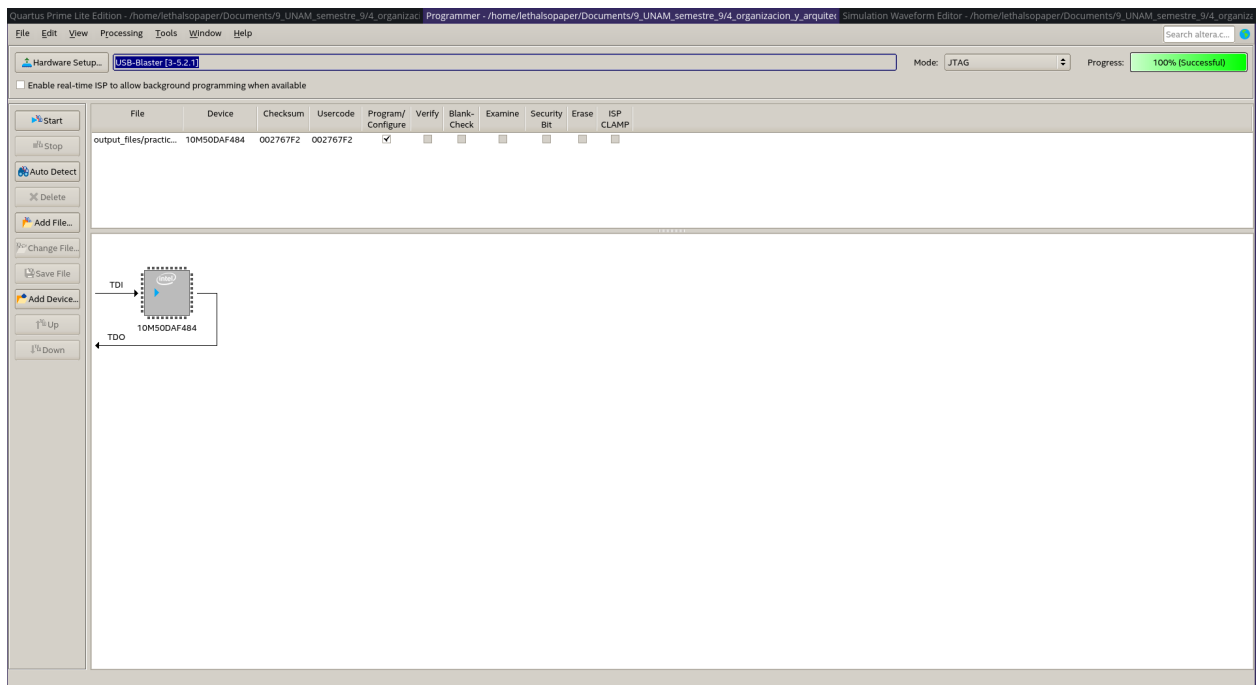


Figure 6: Programación del circuito en la FPGA DE10-Lite.

# Análisis de Resultados

## Resultados en la De10-Lite

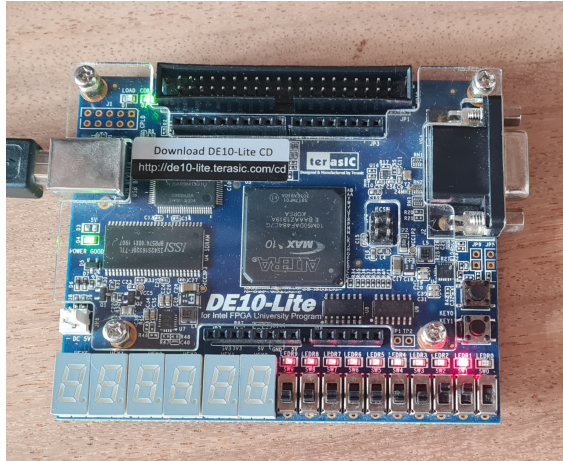


Figure 7: Estado 0( $S1=1$ ,  $S0=0$ ).

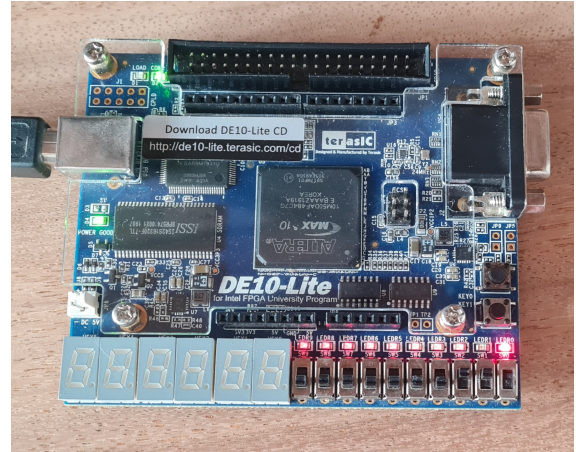


Figure 9: Estado 1( $S1=0$ ,  $S0=1$ ).

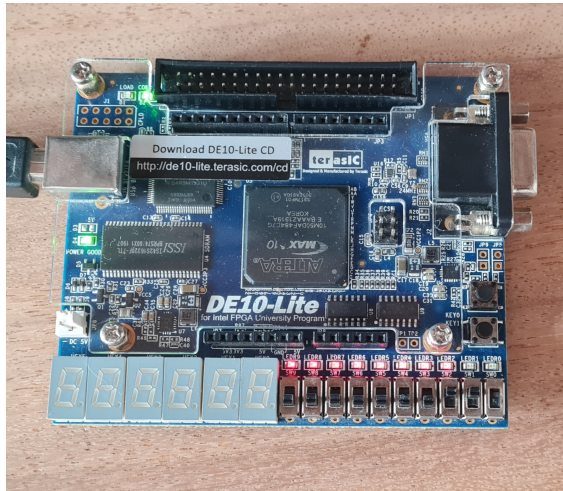


Figure 8: Estado 2( $S1=0$ ,  $S0=0$ ).

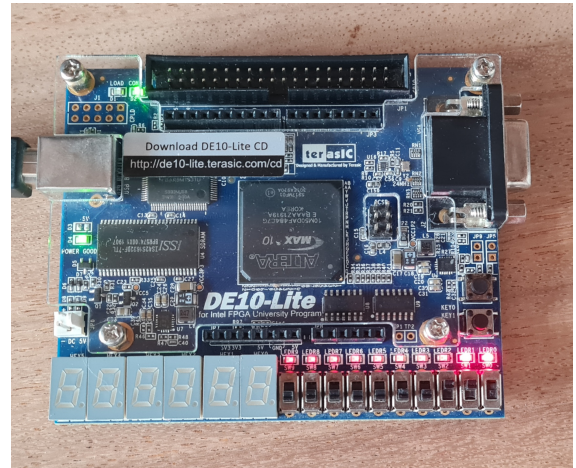


Figure 10: Estado 3( $S1=1$ ,  $S0=1$ ).

## Discusión de Resultados

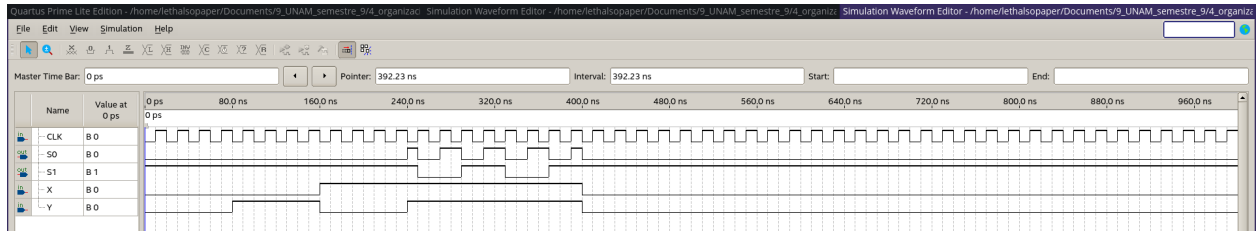


Figure 11: Resultados obtenidos en la FPGA DE10-Lite.

Como podemos ver en la simulación para cada combinación de entradas, el sistema responde correctamente a la carta ASM, mostrando los estados y salidas esperadas. Esto confirma que la implementación de la máquina de estados finitos es correcta y funcional.

## Conclusiones

### 1 Bibliografía

- [1] Esquivel Martinez H. (2021, Noviembre 28). *SISTEMAS MULTIBASE DE DATOS* [Online]. Disponible en:  
<https://sistemasmultibasededatos.blogspot.com/>